⭐ TEXT SUMMARIZATION USING VECTORS — EXTENDED EXPLANATION

Text summarization is the task of producing a shorter version of a document that preserves the most important information.

There are two approaches:

- **Extractive summarization** → select important sentences
- **Abstractive summarization** → generate new sentences

Vector-based summarization falls under **extractive summarization** and relies on **sentence vectors** and **similarity scores**.

🔶 1. Basic Idea Behind Vector-Based Summarization

The core idea is:

Represent each sentence as a vector

→ Using Bag-of-Words (BoW), TF-IDF, or sentence embeddings

Measure importance using vector similarity

→ Sentences most similar to the whole document are selected.

Select top-scoring sentences to form summary

Thus, mathematical representation + ranking = summary.

Scanned with OKEN Scanner

🔶 2. Step-by-Step Process of Vector-Based Summarization

⭐ Step 1: Preprocessing the text

- Convert text to lowercase
- Remove punctuation
- Tokenize sentences
- Remove stopwords
- Stem/lemmatize (optional)

⭐ Step 2: Sentence Vector Representation

There are multiple ways to convert sentences into vectors:

A) Bag-of-Words (BoW) vectors

Each sentence is represented as a vector of word counts.

Example vocabulary of 5 words → sentence vector = size 5.

Limitations: ignores importance of words.

Scanned with OKEN Scanner

B) TF-IDF Vectors

Better representation:

$$TFIDF(w, s) = TF(w, s) \times IDF(w)$$

Where:

- TF = term frequency in sentence
- IDF = inverse document frequency

$$IDF = \log\left(\frac{N}{df(w)}\right)$$

TF-IDF gives more weight to informative words.

C) Sentence Embeddings (Best Method)

Use vector representations from models like:

- Word2Vec
- GloVe
- BERT / SBERT
- spaCy sentence embeddings

Sentence embedding = average (or weighted average) of word embeddings, or a deep model output.

These capture semantic similarity, not just word matching.

Scanned with OKEN Scanner

🔶 3. Step 3: Compute Document Vector

Document vector is computed as:

Option A: Average of all sentence vectors

$$V_{doc} = \frac{1}{n} \sum_{i=1}^{n} V_{sentence_i}$$

Option B: TF-IDF weighted average

More weight to important sentences.

Option C: Principal Component (LSA method)

(Explained below)

🔶 4. Step 4: Sentence Importance Using Similarity

Use **Cosine Similarity**:

$$\text{similarity}(A, B) = \frac{A \cdot B}{\|A\| \, \|B\|}$$

Meaning:

- If similarity close to 1 → sentences aligns with document meaning
- If similarity near 0 → unrelated sentence

Each sentence gets a score:

$$score_i = similarity(V_{sentence_i}, V_{doc})$$

Top-K scored sentences form the summary.

Scanned with OKEN Scanner

🔶 5. Step 5: Redundancy Removal (Optional)

To avoid repeated ideas:

- Compute similarity between candidate sentences
- Remove those with similarity > threshold (0.8)
- Keep diverse sentences only

This improves quality of extractive summary.

⭐ 6. Vector-Based Summarization Using LSA (Latent Semantic Analysis)

(Advanced topic—very good to write in exam)

LSA = SVD (Singular Value Decomposition) applied to TF-IDF matrix.

Steps:

1. Build TF-IDF matrix (sentences × words)
2. Apply SVD:

$$A = U\Sigma V^T$$

3. Choose top-k singular values (k = summary length)
4. Rank sentences by their contribution to top singular vectors

LSA captures:

- Hidden semantic patterns
- Topic structure
- Global theme of document

Thus, top scoring sentences = most important sentences.

Scanned with OKEN Scanner

⭐ 7. Python Implementation Outline (Must Write in Exam)

markdown Copy code

```
1. Read and preprocess text
2. Split into sentences
3. Convert sentences → vectors (TF-IDF or embeddings)
4. Compute document vector
5. Compute cosine similarity for each sentence
6. Rank sentences by similarity score
7. Select top K sentences
8. Arrange in original order → summary
```

Libraries involved:

- NLTK
- spaCy
- scikit-learn (TF-IDF, cosine similarity)
- NumPy

⭐ 8. Example (Short Illustrative)

Document:
"The internet has changed how we communicate. Social media is now widely used. Online tools support communication globally."

Sentence vectors computed → document vector created → similarity scores:

Sentence	Similarity	Result
Sent 1	0.91	selected
Sent 2	0.84	selected
Sent 3	0.55	optional

Thus summary picks sentences 1 and 2.

Scanned with OKEN Scanner

⭐ 9. Advantages of Vector-Based Summarization

✔ Simple to implement

✔ Fast and scalable

✔ Works for any language

✔ TF-IDF + cosine similarity gives good baseline

✔ Embedding-based summarization captures semantics

⭐ 10. Limitations

✘ Extractive → cannot rewrite sentences

✘ May produce disjoint sentences

✘ TF-IDF fails on very long documents

✘ Embedding-based methods depend on pretrained models

⭐ 11. FINAL EXAM CONCLUSION

Text summarization using vectors is an extractive technique that represents sentences numerically using TF-IDF or embeddings. Sentence importance is determined by cosine similarity between sentence vectors and the document vector. High-scoring sentences form the final summary. This method is simple, efficient, and widely used as a baseline for modern summarization systems.

Scanned with OKEN Scanner

⭐ TextRank Intuition — VERY CLEAR EXPLANATION

TextRank is an **unsupervised, graph-based ranking algorithm** for extractive text summarization.
Its intuition is based on a simple idea:

> **A sentence is important if many other important sentences are similar to it.**

This creates a *mutual reinforcement* cycle, very similar to how Google's PageRank works for webpages.

🔶 1. Sentences Behave Like Webpages

Think of each sentence in a document as a **node** in a network.

- If sentence A shares content with sentence B (same words, same concepts), draw an edge between them.
- The more similar they are, the stronger the edge.

Thus:
Sentences = nodes
Sentence similarity = edges
Document = a graph of meaning connections

🔶 2. Importance Comes from Connections

TextRank assumes:

> **An important sentence is one that is connected to many other sentences, especially other important ones.**

This creates a recursive definition of importance:

- A sentence gets a higher score if it is similar to many sentences.
- Those sentences themselves get high scores if *they* are similar to others.

This recursive scoring is what PageRank originally used for ranking web pages.

Scanned with OKEN Scanner

🔶 3. Similarity Instead of Hyperlinks

In webpages, links indicate importance.

In TextRank:

- Similarity = “semantic link” between sentences
- Cosine similarity over TF-IDF or embeddings determines edge weights

If two sentences share information, they are likely covering important parts of the document.

🔶 4. TextRank Emphasizes Central Sentences

The graph structure naturally highlights:

- Sentences that summarize the main topic
- Sentences that overlap with many subtopics
- Sentences located at semantic “hubs” of the document

Thus, **TextRank extracts sentences that best represent the entire document**, not just frequent sentences.

🔶 5. Why TextRank Works So Well?

✔ It captures global context

Because ranking depends on **all** other sentences.

✔ It ignores grammar & syntax

Only similarity matters → works for any language.

✔ It is unsupervised

No labeled data needed → perfect for summarizing new or unknown documents.

✔ It is simple but powerful

Sentences that strongly “connect” with others end up with the highest scores.

Scanned with OKEN Scanner

🔶 6. Summary of the Intuition (Write This in Exam)

TextRank ranks sentences based on how much they "support" each other.

A sentence becomes important if it is similar to many other important sentences.

This creates a network where importance flows across sentences, just like PageRank flows across webpages.

The top-ranked sentences form the final summary.

⭐ PERFECT ONE-LINE INTUITION (Write at end)**

"TextRank finds the most central sentences by treating a document as a graph where similarity acts like a vote of confidence among sentences."

Scanned with OKEN Scanner

Different topic-modeling techniques — detailed explanation

Below is an in-depth, practical, and exam-ready explanation of major topic-modeling methods. For each method I give: **intuition → mathematical model / algorithm → implementation notes & hyperparameters → strengths & weaknesses → practical tips & use cases**. Use this as a study/reference sheet.

---

1. Latent Semantic Analysis (LSA / LSI)

Intuition

LSA (Latent Semantic Analysis) finds latent concepts (topics) by reducing the dimensionality of a term–document matrix. It groups words that co-occur across documents into orthogonal components; each component approximates a "topic".

Math / algorithm

1. Build a term-document matrix A (rows = terms, cols = documents). Use TF or TF–IDF weighting.
2. Compute Singular Value Decomposition (SVD):

$$A = U\Sigma V^\top$$

3. Keep top-k singular values/vectors (rank-k approximation):

$$A_k = U_k \Sigma_k V_k^\top$$

- Columns of U_k relate terms → topic directions.
- Rows of $V_k^\top$ give document coordinates in topic space.

Implementation notes & hyperparams

- Compute TF–IDF first (scikit-learn `TfidfVectorizer`); then use `TruncatedSVD` (scikit) for large corpora.
- Choose k (number of topics / dimensions) by cross-validation or scree plot. Typical k: 50–300 depending on corpus.
- SVD yields negative values — interpretation: components are linear combinations, not probabilities.

Scanned with OKEN Scanner

Strengths

- Fast, simple, deterministic.
- Good at capturing synonymy (words used in similar contexts).

Weaknesses

- Components can be hard to interpret (negative weights).
- Not a generative probabilistic model.
- Loses interpretability for small k if topics mix.

Practical tips

- Preprocess carefully (stopwords, lemmatize).
- Inspect top-weighted words in each component (largest absolute weights) to interpret topics.
- Use for IR (latent semantic indexing) to improve search.

Scanned with OKEN Scanner

2. Latent Dirichlet Allocation (LDA)

Intuition

LDA is a probabilistic generative model: each document is a mixture of latent topics; each topic is a distribution over words. It explains how the observed words in documents might be generated from hidden topics.

Generative model (plates)

For each document d:

1. Draw topic proportions $\theta_d \sim \text{Dirichlet}(\alpha)$.
2. For each word position n in document d:
   - Draw topic $z_{d,n} \sim \text{Categorical}(\theta_d)$.
   - Draw word $w_{d,n} \sim \text{Categorical}(\beta_{z_{d,n}})$, where β_k is topic k's word distribution (drawn from Dirichlet(η) in a full Bayesian view).

Inference / estimation

- Common algorithms: collapsed Gibbs sampling, Variational Bayes (coordinate ascent / stochastic VB).
- Output: per-topic word probabilities $P(w|topic)$ and per-document topic mixtures $P(topic|doc)$.

Implementation notes & hyperparams

- Libraries: `gensim` (Gensim LDA with online learning), scikit-learn (`LatentDirichletAllocation`), MALLET (Gibbs sampling, often better topics).
- Hyperparameters:
  - K: number of topics (must choose).
  - α (document-topic prior) — lower α → sparser topic mixtures per doc.
  - η (topic-word prior) — lower η → sparser topics (fewer words concentrated).
- Iterations, passes, chunksize (online) control convergence.

Strengths

- Interpretable topics (probabilistic, words have non-negative probabilities).
- Handles polysemy via mixture modeling.
- Well-studied, many tools and diagnostics.

Scanned with OKEN Scanner

Weaknesses

- Needs K chosen in advance (though coherence can guide).
- Sensitive to preprocessing (stopwords, rare words).
- Short documents (tweets) make fitting harder (document sparsity).

Practical tips

- Use coherence metrics (UMass, c_v) and stability to choose K.
- Try MALLET's Gibbs sampler if faster convergence / cleaner topics needed.
- Set `alpha` and `eta` (or `doc_topic_prior` and `topic_word_prior`) not default — grid search or heuristics (e.g., `alpha = 50/K`).

3. Non-negative Matrix Factorization (NMF)

Intuition

NMF factorizes a non-negative term-document matrix into non-negative factors W and H where $A \approx WH$. Because factors are non-negative, topics are additive combinations of words (interpretable).

Math / algorithm

Given $A \in \mathbb{R}_{\geq 0}^{m \times n}$, find $W \in \mathbb{R}_{\geq 0}^{m \times k}$, $H \in \mathbb{R}_{\geq 0}^{k \times n}$ minimizing $\|A - WH\|_F$ (Frobenius norm) or KL divergence. Columns of W ~ topic word weights; rows of H ~ document topic weights.

Implementation notes & hyperparams

- Use TF–IDF or raw counts (scikit-learn `NMF`).
- Hyperparams: `n_components` (k topics), initialization (`nndsvd` recommended), regularization, max iterations.
- NMF uses multiplicative update or coordinate descent solvers.

Strengths

- Topics easy to interpret (non-negative).
- Often produces sharper topics than LSA.
- Good when topics combine additively.

Scanned with OKEN Scanner

Weaknesses

- Not probabilistic (no explicit generative story).
- Sensitive to initialization and scale of data.

Practical tips

- Initialize with Nonnegative Double SVD (`nndsvd`) for faster convergence.
- Use TF–IDF input for topic distinctiveness.
- Compare top words across components.

---

4. Probabilistic Latent Semantic Analysis (pLSA)

Intuition

pLSA models document-word co-occurrence with a latent topic variable, similar to LDA, but it does not place a prior on document-topic distributions. It models each document's topic mixture as free parameters.

Model

- $P(w, d) = \sum_z P(z)P(d|z)P(w|z)$ – alternative factorizations exist.
- Document-topic probabilities are parameters learned via EM.

Implementation notes

- pLSA uses EM for parameter estimation.
- Overfits as number of documents grows (document-topic parameters grow), which motivated LDA's Bayesian prior.

Strengths

- Simpler conceptually than LDA; sometimes easier to implement.

Weaknesses

- Overfitting; lacks a proper generative model for unseen documents (cannot naturally infer topic mix for new docs without retraining).

Practical tip

- Historically important; LDA replaced it for practical use.

Scanned with OKEN Scanner

5. Hierarchical Dirichlet Process (HDP) — non-parametric Bayesian

Intuition

HDP lets the data determine the number of topics. It is a hierarchical Bayesian model using Dirichlet Processes so the model can use as many topics as needed.

High-level model

- Each document has a Dirichlet Process mixing measure centered on a global base measure that itself is a DP — this ties topics across documents but allows an unbounded number of topics.

Implementation notes

- Libraries: `gensim` has an `HdpModel`. There are also Bayesian inference libraries (PyMC3, BNP packages).
- Inference is often done via Gibbs sampling or variational methods.

Strengths

- No need to preset K.
- Flexibility for corpus with unknown topic complexity.

Weaknesses

- More complex inference & slower than LDA.
- Hyperparameter tuning (concentration parameters) affects number of topics.

Practical tips

- Use when you genuinely don't want to guess K; otherwise LDA + model selection may be simpler.

Scanned with OKEN Scanner

KTU On the Go (CSE) | IAN syllabus overview | Google Gemini
chatgpt.com/c/6932f387-eb94-8323-8606-1b
ChatGPT 5.1 Share Add people

Latent Semantic Modelling / Indexing (LSA / LSI) — Full, exam-ready explanation + Python how-to

Below is a complete, exam-perfect treatment covering:

- What LSA / LSI is (introduction)
- Intuition for Singular Value Decomposition (SVD)
- How SVD is applied to NLP (LSA / LSI pipeline & math)
- Practical Python implementation (clean, copy-pasteable code)
- How to pick dimensionality, interpret components, advantages/limitations, evaluation and tips

Write this in the exam (7–10 marks) or use the code directly for experiments.

1. LSA / LSI — Introduction (What & Why)

Latent Semantic Analysis (LSA), also called **Latent Semantic Indexing (LSI)** when used for information retrieval, is a linear algebra method that uncovers *latent* (hidden) semantic structure in a term–document matrix by reducing dimensionality.

Problem it solves

- Raw term–document matrices are high-dimensional and sparse (many zeros).
- Synonymy: different words with similar meanings appear in similar documents.
- Polysemy: same word has different meanings in different contexts.

LSA idea

- Build a matrix A where rows = terms, cols = documents (often TF–IDF weighted).
- Factorize A with SVD and keep only the top k singular values/vectors to get a low-rank approximation A_k.
- The low-rank space captures major co-occurrence patterns → "topics" or latent concepts.
- Use the reduced representations for retrieval, clustering, similarity, or visualization.

When used for retrieval (search), this approach is called **LSI**: queries and documents are projected in the same latent space and ranked by similarity

Scanned with OKEN Scanner

2. Singular Value Decomposition — Intuition (SVD)

SVD statement (matrix factorization):

For any real matrix $A \in \mathbb{R}^{m \times n}$, SVD gives:

$$A = U\Sigma V^\top$$

- U is $m \times r$: orthonormal columns (left singular vectors) — directions in **term** space.
- Σ is $r \times r$ diagonal with singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$.
- V is $n \times r$: orthonormal columns (right singular vectors) — directions in **document** space.
- r = rank(A).

Intuition:

- SVD finds orthogonal axes that capture maximal variance (energy) of the matrix.
- The singular values measure how much information each axis captures.
- Truncating to top k singular values gives the **best** rank-k approximation (in least squares sense):
  $A_k = U_k\Sigma_k V_k^\top$ minimizes $\|A - B\|_F$ over matrices B of rank k.
- In NLP terms, the top components capture dominant semantic patterns (topics); noise and idiosyncratic variations are in smaller singular values.

Geometric picture: project high-dimensional term vectors or document vectors into a compact k-dimensional latent space where semantically similar words/documents lie near each other.

Scanned with OKEN Scanner

3. Applying SVD to NLP — LSA / LSI math & pipeline

3.1 Term–Document matrix construction

- Build matrix A where A_{ij} = weight of term i in document j.
- Common weighting: **TF–IDF** (better than raw counts for semantics).
- Optionally apply stopword removal, lemmatization, phrase detection (bigrams).

3.2 SVD and truncation

Compute:

$$A = U\Sigma V^\top$$

Keep top k singular values/vectors:

$$A_k = U_k \Sigma_k V_k^\top$$

- U_k: $m \times k$ — term→topic matrix (term loadings).
- Σ_k: $k \times k$ — strengths of latent dimensions.
- V_k: $n \times k$ — document coordinates in latent space.

3.3 Representations

- **Term vector in latent space:** row i of $U_k\Sigma_k$.
- **Document vector in latent space:** row j of $V_k\Sigma_k$ (or equivalently columns of V_k scaled by singular values).
- Query q (bag of words / TF–IDF) is projected into latent space: $q_k = q^\top U_k \Sigma_k^{-1}$ (or appropriately using the decomposition depending on conventions) — then compare q_k with document vectors (cosine similarity).

3.4 Use cases

- **Information retrieval (LSI):** project docs & queries into latent space; rank docs by cosine similarity.
- **Document clustering / topic discovery:** cluster rows of $V_k\Sigma_k$ or use top words from components.
- **Dimensionality reduction** before classification or visualization.

Scanned with OKEN Scanner

4. What does a latent dimension mean?

- Each latent dimension is a direction in term space that combines correlated words.
- Top words with large positive (or negative) weights on a component indicate the semantic theme.
- Unlike probabilistic topics (LDA), LSA components are orthogonal and can have positive/negative weights (interpret by absolute magnitude, sign gives opposing association).

5. How to choose k (number of latent dimensions)

Practical guidelines:

- Typical ranges: **50–300** for medium/large corpora (documents in thousands).
- Use a **scree plot** of singular values and choose elbow point.
- Evaluate downstream task performance (retrieval accuracy, clustering coherence) on validation set.
- If k too small → underfitting (topics too coarse). Too large → overfitting / noisy patterns.

6. Advantages & limitations

Advantages

- Simple, deterministic, fast with optimized SVD (TruncatedSVD, randomized SVD).
- Captures synonymy (grouping similar terms).
- Useful baseline and for retrieval (LSI).

Limitations

- Linear method: cannot capture complex nonlinear semantics.
- Components can be hard to interpret; negative weights complicate “topic” interpretation.
- Not probabilistic: no explicit generative model like LDA.
- Sensitive to preprocessing and choice of k.
- TF–IDF input important — raw counts usually worse.

↓

Scanned with OKEN Scanner